Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Дисциплина «Системы компьютерной обработки изображений»

Отчёт по лабораторной работе № 1
Хромакей

Вариант № 5

Группа: P3418

Выполнил: Баширов А. К.

Проверил:
Денисов А. К.

Санкт-Петербург
2026

СОДЕРЖАНИЕ

Введение 3
1 Теоретическая база 4
1.1 Цветовая модель и условие замены 4
1.2 Маска и совмещение изображений 4
1.3 Трудоёмкость 4
2 Описание реализации 5
2.1 Нативный алгоритм 5
2.2 Алгоритм на OpenCV 5
2.3 Команды запуска 5
3 Результаты и тестирование 7
3.1 Исходные данные и маска 7
3.2 Влияние границ интервала 8
3.3 Сравнение быстродействия 9
Вывод 11

ВВЕДЕНИЕ

Цель работы — реализовать хромакей по заданному цветовому интервалу двумя способами и сравнить их быстродействие. Первый способ использует готовые функции библиотеки OpenCV. Второй выполняет ту же замену нативным циклом на Python.

Постановка задачи. Даны два изображения и цвет. Цвет задаётся не одной точкой, а интервалом, границы которого вводит пользователь. Каждый пиксель первого изображения, попавший в интервал, заменяется пикселем второго изображения с теми же координатами. Остальные пиксели первого изображения сохраняются.

Работа выполнена на Windows в Python 3.9.0. Использованы OpenCV 5.0.0.93, NumPy 2.0.2 и Pillow 11.3.0. Исходный код и результаты прогонов размещены в репозитории https://github.com/jarashko/akg. Демонстрационные кадры сформированы программой: фигура на зелёном поле и второе изображение с небом и землёй, оба размера 960×540.

1 ТЕОРЕТИЧЕСКАЯ БАЗА

1.1 ЦВЕТОВАЯ МОДЕЛЬ И УСЛОВИЕ ЗАМЕНЫ

Цифровое изображение — прямоугольная сетка пикселей ширины W и высоты H. В модели RGB у пикселя три канала: красный R, зелёный G и синий B. В программе каждый канал — целое число от 0 до 255.

Пользователь задаёт нижнюю границу $L = (L_R, L_G, L_B)$ и верхнюю границу $U = (U_R, U_G, U_B)$. Пиксель переднего плана (R, G, B) заменяется, если выполнены все три неравенства одновременно:

$$L_R \le R \le U_R, \quad L_G \le G \le U_G, \quad L_B \le B \le U_B.$$

Границы входят в интервал. Если хотя бы один канал лежит вне своего отрезка, пиксель переднего плана сохраняется. В пространстве RGB такое условие задаёт прямоугольный параллелепипед, выровненный по осям. Если пользователь перепутал нижнюю и верхнюю границы, программа по каждому каналу отдельно берёт меньшее значение как низ интервала, а большее — как верх.

Зелёный и синий экраны применяют потому, что в оттенках кожи красный канал обычно заметно выше зелёного и синего. Интервал можно выбрать так, чтобы он покрыл фон и не задел лицо. В работе используется зелёный экран.

1.2 МАСКА И СОВМЕЩЕНИЕ ИЗОБРАЖЕНИЙ

Маска — одноканальное изображение того же размера. Значение 255 означает, что пиксель входит в интервал и в результат берётся второе изображение. Значение 0 означает, что пиксель остаётся от переднего плана. На иллюстрациях 255 показано белым цветом, 0 — чёрным.

Результат имеет размер переднего плана. Если второе изображение отличается по размеру, оно приводится к ширине W и высоте H билинейной интерполяцией. После этого пиксель с координатами (x, y) всегда сопоставляется пикселю второго изображения с теми же координатами.

Замена в данной работе жёсткая: полупрозрачность, сглаживание границы и подавление цветового ореола не выполняются. Пиксель целиком берётся из одного из двух кадров.

1.3 ТРУДОЁМКОСТЬ

На каждый пиксель приходится постоянное число сравнений, окрестность пикселя не просматривается. Поэтому оба алгоритма имеют временную сложность $O(WH)$. Дополнительная память также линейна по числу пикселей: хранятся исходные кадры, маска и результат.

2 ОПИСАНИЕ РЕАЛИЗАЦИИ

2.1 НАТИВНЫЙ АЛГОРИТМ

Функция `chroma_key_native` обходит пиксели средствами Python. Для уже выровненных по размеру кадров решение принимается так:

```
для каждого пикселя (x, y):
    если Lr ≤ R ≤ Ur и Lg ≤ G ≤ Ug и Lb ≤ B ≤ Ub:
        результат[x, y] = второе изображение[x, y]
    иначе:
        результат[x, y] = передний план[x, y]
```

Чтение и запись буфера выполняет Pillow. Проверка цвета и выбор пикселя выполняются интерпретатором CPython, поэтому время определяется числом итераций цикла.

2.2 АЛГОРИТМ НА OPENCV

Функция `chroma_key_opencv` получает тот же результат без цикла на Python.

1. `cv2.inRange` строит маску: 255 внутри интервала и 0 снаружи.
2. `cv2.bitwise_not` меняет эти значения местами и тем самым выделяет сохраняемую часть переднего плана.
3. `cv2.bitwise_and` по прямой маске оставляет пиксели второго изображения, а по инвертированной маске — пиксели переднего плана. Остальные пиксели обнуляются.
4. `cv2.add` складывает две части.

Маски дополнительные: где одна равна 255, другая равна 0. Сложение не достигает ограничения 255 и совпадает с копированием нужного пикселя. После каждого сравнительного прогона массивы двух реализаций проверяются целиком. При различии хотя бы одного пикселя программа останавливается.

2.3 КОМАНДЫ ЗАПУСКА

Зависимости устанавливаются из каталога проекта:

```
python -m pip install -r requirements.txt
```

Сводный пример, маска, три интервала и измерение времени строятся одной командой:

```
python chroma_key.py --demo
```

Файлы сохраняются в каталог `results`. Для отдельного прогона рабочего интервала в PowerShell используется команда:

```
python chroma_key.py --fg results/foreground.png `
    --bg results/background.png `
    --lower 0,120,0 --upper 80,255,80 `
    --output result.png --mask mask.png --benchmark
```

Параметр `--benchmark` печатает медиану времени обеих реализаций. В замер не входят чтение исходных файлов и запись результата. Перед серией выполняется один прогрев. Узкий и широкий интервалы запускаются теми же аргументами `--lower` и `--upper` со значениями из таблицы 1.

3 РЕЗУЛЬТАТЫ И ТЕСТИРОВАНИЕ

3.1 ИСХОДНЫЕ ДАННЫЕ И МАСКА

Передний план — зелёное поле с горизонтальным градиентом и небольшой вертикальной волной яркости. Поверх поля нарисована фигура. Кожа имеет цвет (235, 190, 150), рубашка — (25, 45, 210), ноги — (50, 45, 55). Зелёный фон укладывается примерно в пределы R от 8 до 56, G от 129 до 251 и B от 12 до 48. Второе изображение содержит небо, солнце и землю.

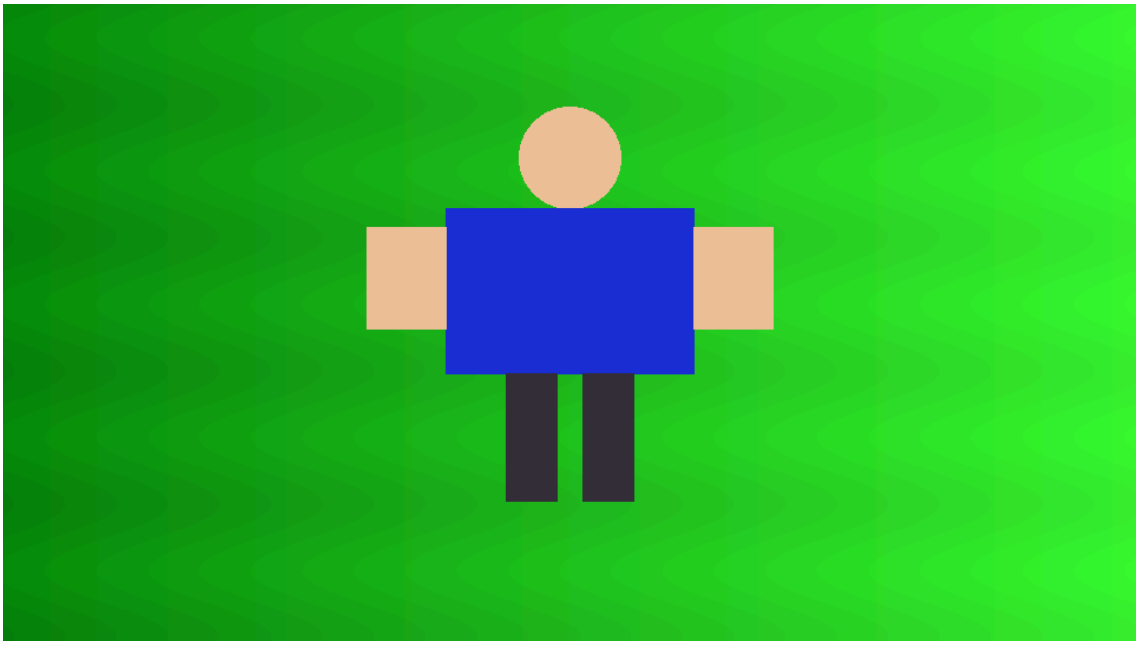


Рисунок 1 — Передний план: фигура на зелёном поле



Рисунок 2 — Второе изображение, которым заменяется цветовой ключ

На рисунке 3 белый цвет соответствует пикселям рабочего интервала $(0, 120, 0)\ldots(80, 255, 80)$, чёрный — пикселям фигуры. Силуэт чёрен, потому что ни кожа, ни рубашка, ни ноги в этот интервал не входят.

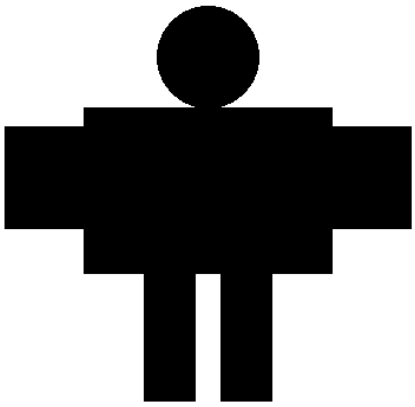


Рисунок 3 — Маска рабочего интервала: белым отмечены заменяемые пиксели

3.2 ВЛИЯНИЕ ГРАНИЦ ИНТЕРВАЛА

Один и тот же кадр обработан тремя интервалами. Результаты сведены в таблице 1 и показаны на рисунке 4.

Таблица 1 — Границы интервала и наблюдаемый результат

Режим	**Нижняя граница**	**Верхняя граница**	**Наблюдение**
Узкий	(0, 200, 0)	(80, 255, 80)	Заменена только яркая правая часть фона
Рабочий	(0, 120, 0)	(80, 255, 80)	Фон заменён, фигура сохранена
Широкий	(0, 0, 0)	(255, 255, 180)	Сохранилась только синяя рубашка

Рабочий интервал покрывает зелёный фон: красный канал фона не превосходит 56, зелёный не опускается ниже 129, синий не превосходит 48. Кожа отсекается красным каналом 235, рубашка — синим каналом 210, ноги — зелёным каналом 45.

Узкий интервал дополнительно требует $G \geq 200$. Такое значение есть только на яркой правой части градиента, поэтому слева зелёный экран остаётся. Граница волнистая: к горизонтальному градиенту добавлена небольшая вертикальная волна зелёного канала, и линия $G = 200$ не является прямой.

Широкий интервал поднимает верхнюю границу синего до 180. Кожа с $B = 150$ и ноги с $B = 55$ попадают внутрь интервала и исчезают. У рубашки $B = 210$, поэтому она остаётся на фоне второго изображения.

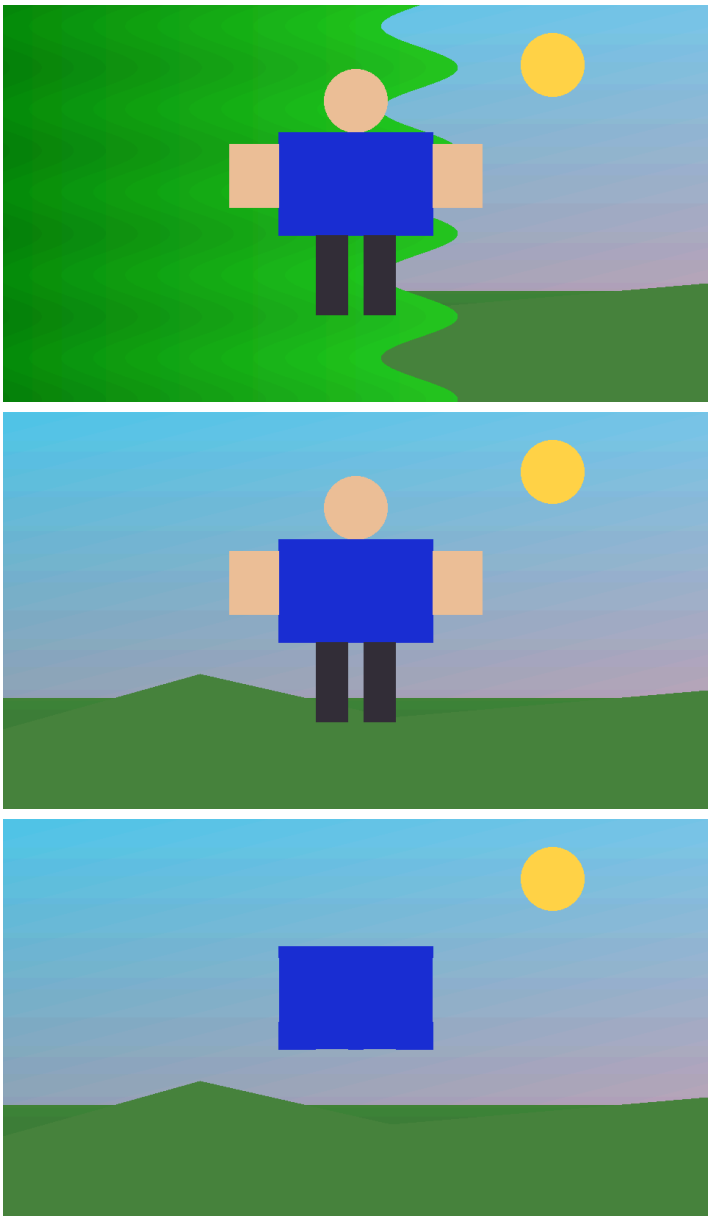


Рисунок 4 — Сверху вниз: узкий, рабочий и широкий интервалы

Нативный цикл и OpenCV на всех трёх интервалах, а также на трёх разрешениях из следующего пункта, дали побитово одинаковые изображения.

3.3 СРАВНЕНИЕ БЫСТРОДЕЙСТВИЯ

Измерялась только замена пикселей. Для 320×240 медиана взята по 5 прогонам Python и 40 прогонам OpenCV, для 640×480 — по 3 и 30, для 1280×720 — по 2 и 20. Результаты приведены в таблице 2 и на рисунке 5.

Таблица 2 — Медианное время хромакея

Разрешение	Пикселей	Python, мс	OpenCV, мс	Ускорение
320×240	76 800	33,35	0,73	45,9×
640×480	307 200	139,12	3,50	39,7×
1280×720	921 600	421,18	10,49	40,1×

От 320×240 к 640×480 число пикселей увеличивается в 4 раза. Время Python выросло в 4,17 раза, время OpenCV — в 4,8 раза. От 640×480 к 1280×720 пикселей становится в 3 раза больше, а время — в 3,03 и 3,00 раза. Зависимость близка к линейной и согласуется с оценкой $O(WH)$.

OpenCV быстрее примерно в 40 раз: обход памяти выполняется кодом библиотеки на C++, а нативный вариант принимает решение по каждому пикселю в интерпретаторе. На отдельном кадре 960×540 получено 235,26 мс против 5,37 мс, ускорение составило 43,8 раза. На графике шкала времени логарифмическая, подписи над столбцами даны в миллисекундах. Повторный запуск `python chroma_key.py --demo` перезаписывает изображения и файл `results/timings.txt`. Абсолютные миллисекунды зависят от компьютера, линейный рост и кратность ускорения сохраняются.

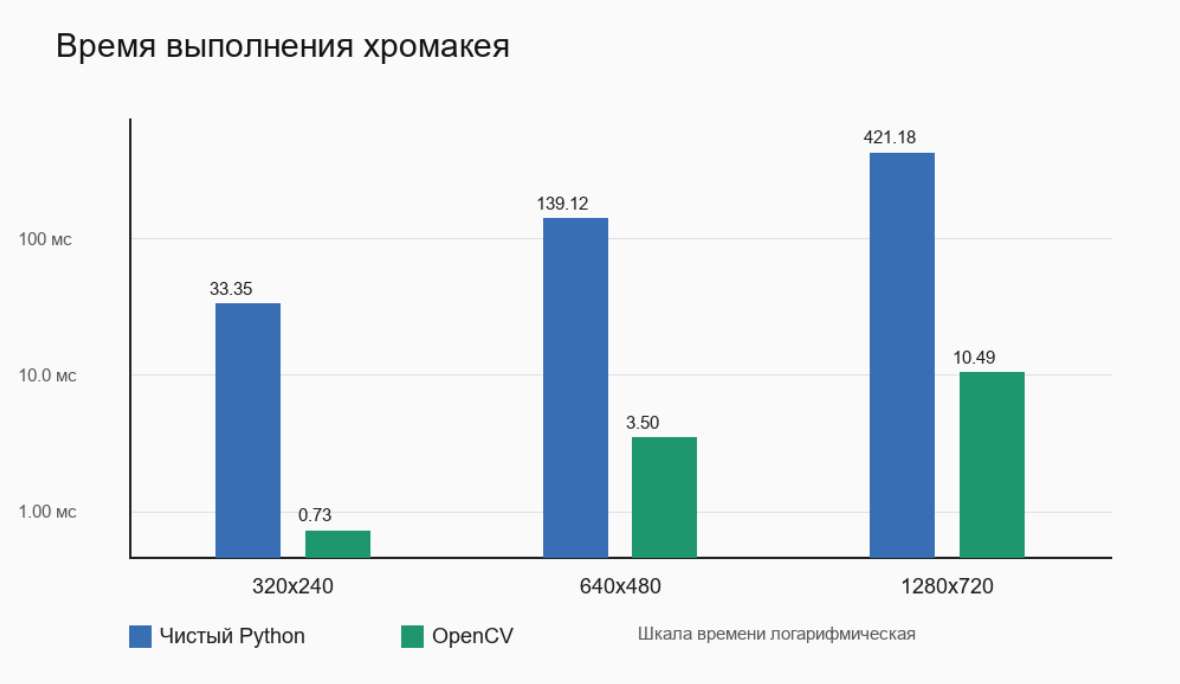


Рисунок 5 — Время нативной реализации и OpenCV при трёх разрешениях

ВЫВОД

1. Реализован хромакей варианта 5: пиксели переднего плана, попавшие в интервал RGB, заменяются пикселями второго изображения.
2. Нативный цикл и связка функций OpenCV дали одинаковый результат на трёх интервалах и трёх разрешениях.
3. Время обеих реализаций растёт пропорционально числу пикселей.
4. На проведённых измерениях OpenCV быстрее нативного цикла примерно в 40 раз.
5. Рабочий интервал $(0, 120, 0)\ldots(80, 255, 80)$ заменяет зелёный фон и сохраняет фигуру. Слишком узкий интервал оставляет часть экрана, слишком широкий захватывает цвета объекта.